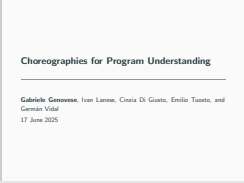
Choreographies for Program Understanding

Gabriele Genovese, Ivan Lanese, Cinzia Di Giusto, Emilio Tuosto, and Germán Vidal

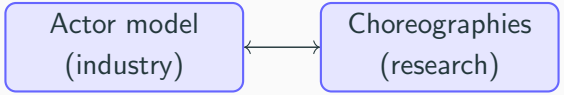
17 June 2025

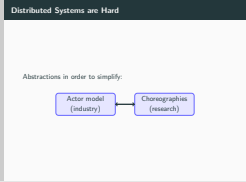


Good afternoon, everyone. Today I'm presenting our work titled "Choreographies for Program Understanding," developed in collaboration with Ivan Lanese, Cinzia Di Giusto, Emilio Tuosto, and Germán Vidal. This work is about analyzing distributed systems from a global perspective, even when we only have access to the local code with an ad-hoc extraction techniques. Let's start by looking at the big picture of distributed systems.

Distributed Systems are Hard

Abstractions in order to simplify:



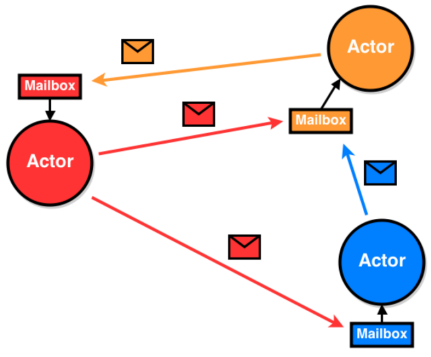


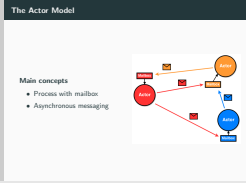
Distributed systems are complex by nature. To manage this complexity, we rely on abstractions that help structure and reason about interactions. In industry, the actor model is commonly used—think Erlang or Akka. Meanwhile, in academia, we focus on choreographies, which give a global view of the system behavior. In this talk, I'll show how we bridge these two views.

The Actor Model

Main concepts

- Process with mailbox
- Asynchronous messaging



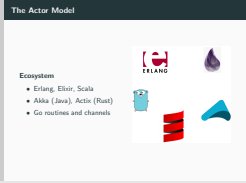


Let's start with the actor model. Each actor is an independent process with its own mailbox. They interact purely by sending and receiving messages asynchronously. This model scales well and fits naturally into distributed environments.

The Actor Model

Ecosystem

- Erlang, Elixir, Scala
- Akka (Java), Actix (Rust)
- Go routines and channels

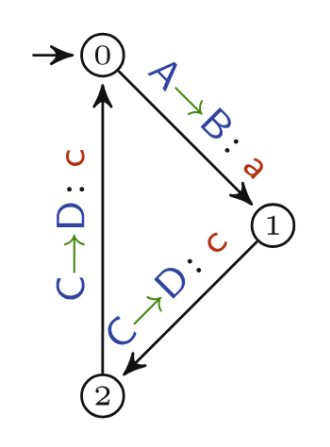


The actor model has strong practical support. Languages like Erlang, Elixir, and Scala adopt it directly. Frameworks like Akka for Java and Actix for Rust are built around it. Even Go, although not actor-based, has a similar concurrency model using goroutines and channels.

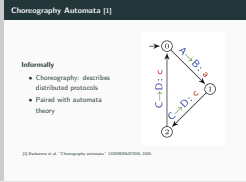
Choreography Automata [1]

Informally

- Choreography: describes distributed protocols
- Paired with automata theory



[1] Barbanera et al. "Choreography automata." COORDINATION, 2020.



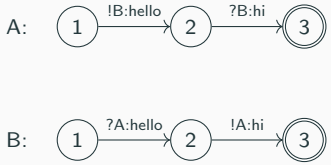
In research, we often take a more global view of communication. A choreography describes all the participants and their interactions from a central perspective. By pairing this with automata theory, we can rigorously define and analyze these protocols. This idea was formalized in "Choreography Automata" by Barbanera et al.

Global vs Local View

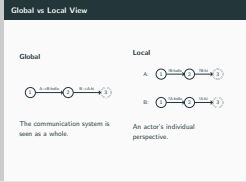
Global

The communication system is seen as a whole.

Local



An actor's individual perspective.



Choreographies offer both a global view—where we see all interactions—and local views, where each participant sees only their own role. For example, if A sends "hello" to B and B replies "hi", the global view captures the full exchange. Locally, A just sees "send hello", and B just sees "receive hello, then send hi". Translating between these two views is key to our work.

Top-Down Approach

Steps:

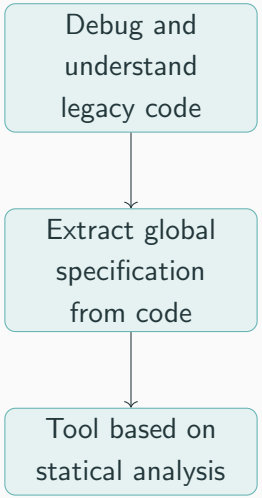
1. Write the global specification
2. Project to obtain the local specification
3. Write the local functions that respect the specification

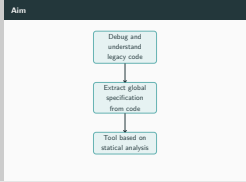
Problem: no way to include existing code and architecture.

Top-Down Approach
Steps:
1. Write the global specification
2. Project to obtain the local specification
3. Write the local functions that respect the specification
Problem: no way to include existing code and architecture.

One classical approach is top-down: You write a global choreography, project it to get local specifications, and implement those locally. However, this doesn't work well if you're dealing with existing systems or legacy code—you can't easily inject a top-down design after the fact.

Aim





Our aim is different: We want to understand and debug legacy systems. The idea is to extract a global view from already written code. We're building a tool to do this using static analysis, starting from real programs.

Bottom-up approach

Extraction steps:

1. Analyze an input source code
2. Create the local view
3. Combine the local view to create an approximation global view

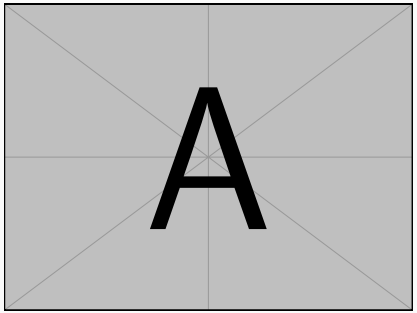
Bottom-up approach
Extraction steps:
1. Analyze an input source code
2. Create the local view
3. Combine the local view to create an approximation global view

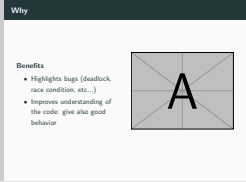
So instead of top-down, we use a bottom-up approach: First, we analyze the code to extract local views. Then, we combine them to build an approximate global view. It's not perfect, but it can already tell us a lot—especially when we care about communication behavior.

Why

Benefits

- Highlights bugs (deadlock, race condition, etc...)
- Improves understanding of the code: give also good behavior





Why go through all this effort? Well, the extracted models can reveal bugs like deadlocks or race conditions. But they also show correct behaviors—they help us understand what the code actually does. This is helpful both for developers and educators working on distributed algorithms.

Requirements

- Automatic extraction using bottom-up and over-approximation techniques
- Capture good behaviors and highlight misbehavior
- Support multiple actors
- Target mainstream language
- Simple notation

Requirements
- Automatic extraction using bottom-up and over-approximation techniques
- Capture good behaviors and highlight misbehavior
- Support multiple actors
- Target mainstream language
- Simple notation

To make this useful in practice, we need several things: First, the process should be automatic, using static analysis and over-approximation. It should highlight both good and bad behaviors, support multiple actors, and be applicable to mainstream languages. Finally, the output should be easy to read— something people can reason with.

Problems

- Undecidability: we should decide termination
- Huge descriptions: given the over-approximation approach

Problems
• Undecidability: we should decide termination
• Huge descriptions: given the over-approximation approach

Naturally, there are challenges. One is undecidability: we can't always determine termination or correctness. Another is that over-approximations can lead to very large models, which can be hard to manage. So the trick is to balance precision and scalability.

Motivating Examples

Two example:

Dining Philosophers
(deadlock)

Bank Account
(race condition)

Motivating Examples
Two example:
Dining Philosophers (deadlock)
Bank Account (race condition)

Deadlock

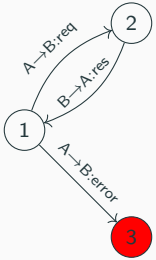


Figure 1: A deadlock can occur if a non-final state without outgoing transitions is present.

Deadlock
Figure 1: A deadlock can occur if a non-final state without outgoing transitions is present.

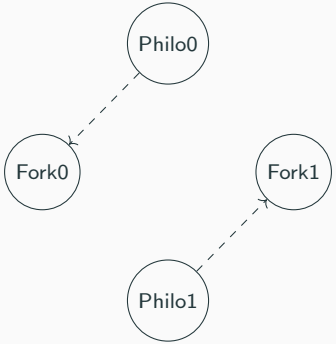
Dining Philosophers example

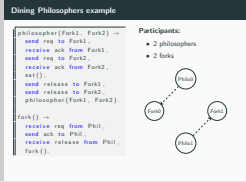
```
philosopher(Fork1, Fork2) →
  send req to Fork1,
  receive ack from Fork1,
  send req to Fork2,
  receive ack from Fork2,
  eat(),
  send release to Fork1,
  send release to Fork2,
  philosopher(Fork1, Fork2).

fork() →
  receive req from Phil,
  send ack to Phil,
  receive release from Phil,
  fork().
```

Participants:

- 2 philosophers
- 2 forks





Dining Philosophers global-view

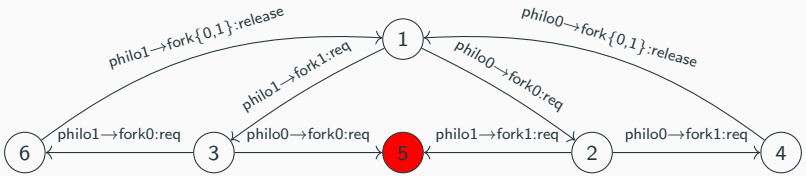


Figure 2: Global view of the dining philosophers example.

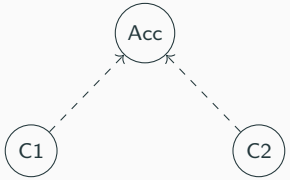
Bank account example

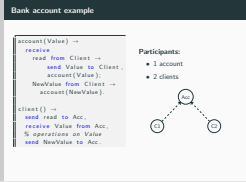
```
account(Value) →
  receive
    read from Client →
        send Value to Client,
        account(Value);
    NewValue from Client →
      account(NewValue).

client() →
  send read to Acc,
  receive Value from Acc,
  % operations on Value
  send NewValue to Acc.
```

Participants:

- 1 account
- 2 clients





Bank account global-view

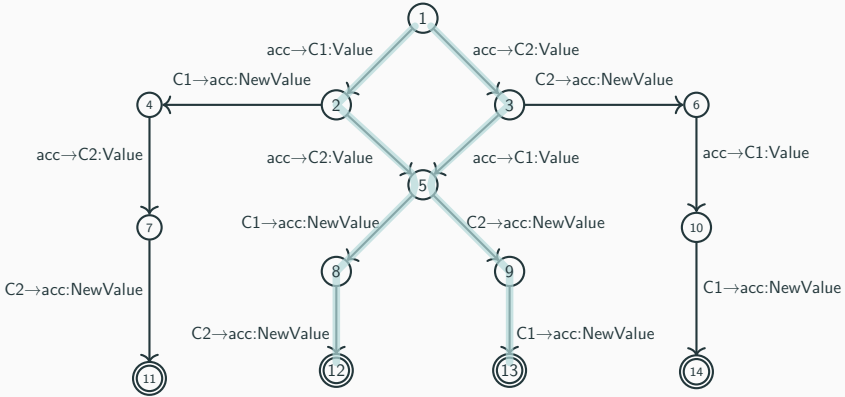
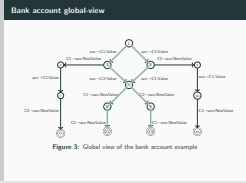


Figure 3: Global view of the bank account example



Conclusion

Summary

- Problem: no way to express legacy code
- Bridge the gap between programming and theoretical frameworks
- Use of bottom-up and over-approx techniques

Future work: create the tool as described (WIP) and evaluate it on real-world examples.

Conclusion
Summary
• Problem: no way to express legacy code
• Bridge the gap between programming and theoretical frameworks
• Use of bottom-up and over-approx techniques
Future work: create the tool as described (WIP) and evaluate it on real-world examples.

Thanks for listening!